Running Infinia on Windows PC with WSL

WSL - “Windows Subsystem for Linux” is Microsoft’s “native“ solution to run Linux environment on a Windows PC. Since WSL 2, it is based on the Windows embedded Hyper-V virtualization, so you can run the entire Infinia stack on your Windows PC, just as on other VMs. For more details about WSL, visit:

> [Windows Subsystem for Linux Documentation | Microsoft Learn]

Before start, please be aware that you will need to recompile and upgrade the built-in Kernel of WSL to Linux 6.6.x and change some global settings of WSL. Those changes are supported by Microsoft but will take effective on all the WSL instances on your PC.

1. Install WSL on your PC. Make sure your are using WSL 2 and Ubuntu (which are default configurations). [Install WSL | Microsoft Learn]
2. Recompile your Linux Kernel by [How to use the Microsoft Linux kernel v6 on WSL2 | Microsoft Learn]
   a. The reason is redsetup will try to collect the NUMA information in /sys/devices/system/node/ but the default WSL 5.15.x kernel disables the NUMA support (so, doesn’t has that sysfs dir). https://github.red.datadirectnet.com/red/red/blob/2aec6036e29439b311a41a5e49d8c47ecece3698/src/discover/red_node.cpp#L208
   b. For the Infinia setup in VM, we do not really require the VM firmware to provide any real NUMA information, which typically doesn’t make sense for VM. The Linux Kernel with NUMA enabled will generate a fake sysfs structure for the none-NUMA firmware (putting all memory in a single node).
   c. In the above guide, make sure your are using the 6.6.y kernel (which currently is the default kernel branch) and the default config file provided by Microsoft. It has NUMA enabled. For example:

```
sudo apt update && sudo apt install build-essential flex bison libssl-dev libelf-dev bc python3 pahole cpio
git clone https://github.com/microsoft/WSL2-Linux-Kernel.git --depth=1 -b linux-msft-wsl-6.6.y
cd WSL2-Linux-Kernel
make -j$(nproc) KCONFIG_CONFIG=Microsoft/config-wsl
sudo make modules_install headers_install
cp arch/x86/boot/bzImage /mnt/c/
```

3. Before rebooting with the new kernel, update /etc/wsl.conf to set a new hostname for your Linux VM.

a. By default, WSL will use the same hostname as your Windows. It may work (not fully verified) but here we will create a new name, for example, wslserver in my case:

```
fengshuo@wslserver:~$ cat /etc/wsl.conf
[boot]
systemd=true

[network]
hostname=wslserver
```

b. For more information about wsl.conf, please reference: [Advanced settings configuration in WSL | Microsoft Learn]() . It will be very useful for below sections also.

4. Before rebooting with the new kernel, create a new file in /etc/modules-load.d/ to load the docker required kernel modules.
   a. The reason is Microsoft don't allow module auto-loading for user compiled kernels. You have to "modprobe" any required modules explicitly: [docker cannot start after installing linux-msft-wsl-6.6.y · Issue #11742 · microsoft/WSL]()
   b. If you have already rebooted your WSL and have docker installed, your docker service will not be able to start. You can manually modproble those modules and restart the docker service by systemctl restart docker.
   c. My examples is /etc/modules-load.d/docker_modules.conf. You may add other modules if you need for your VM.

```
fengshuo@wslserver:~$ cat /etc/modules-load.d/docker_modules.conf
# based on https://github.com/microsoft/WSL/issues/11742
# try to manually install modules for docker.
xt_addrtype
br_netfilter
nft_compat
xt_conntrack
xt_MASQUERADE
xfrm_user

# this is for redsetup only
xt_nat
```

5. Continue following the guide to shutdown the VM: [How to use the Microsoft Linux kernel v6 on WSL2 | Microsoft Learn]()

```
wsl --shutdown
```

6. Continue following the above guide to install the new kernel, with below changes required for Infinia. Be aware the below operations are done on your Windows system (by the GUI "WSL settings" or editing `%USERPROFILE%\.wslconfig`). Don't open any WSL console before this step is done.
   a. Disable the firewall.

b. Disable cgroup v1. --- The Ubuntu's docker service on WSL will use cgroup v2, but the docker-compose installed by Infinia will try to start with cgroup v1. This causes conflict as Infinia services are started after the systemd docker service. Pay attention to the "kernelCommandLine" setting for cgroup as below.

c. Other system tuning needed, for example, memory (not necessary if you only have 16GB on your laptop). Here is my .wslconfig

```
[wsl2]
firewall=false
kernel=C:\\bzImage
kernelCommandLine = cgroup_no_v1=all systemd.unified_cgroup_hierarchy=1
processors=4
```

7. Reopen the WSL console, the VM will be automatically started. Make sure all changes (hostname, kernel, processor, cgroup v2) are valid in the new system. If not, reboot the system again (sometime the WSL instance may start before you finish editing the .wslconfig file).

8. After confirmed, edit /etc/wsl.conf to stop WSL generating the /etc/hosts file.

```
fengshuo@wslserver:~$ cat /etc/wsl.conf
[boot]
systemd=true

[network]
hostname=wslserver
generateHosts = false
```

9. Update your /etc/hosts file to register your IP address, 172.26.121.28 and wslserver in my case. Don't worry about the warning in the first line as we have disabled the auto-generate of this file.

```
fengshuo@wslserver:~$ cat /etc/hosts
# This file was automatically generated by WSL. To stop automatic generation of this file, add the following entry to /etc/wsl.conf:
# [network]
# generateHosts = false
127.0.0.1       localhost
172.26.121.28   wslserver.      wslserver

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

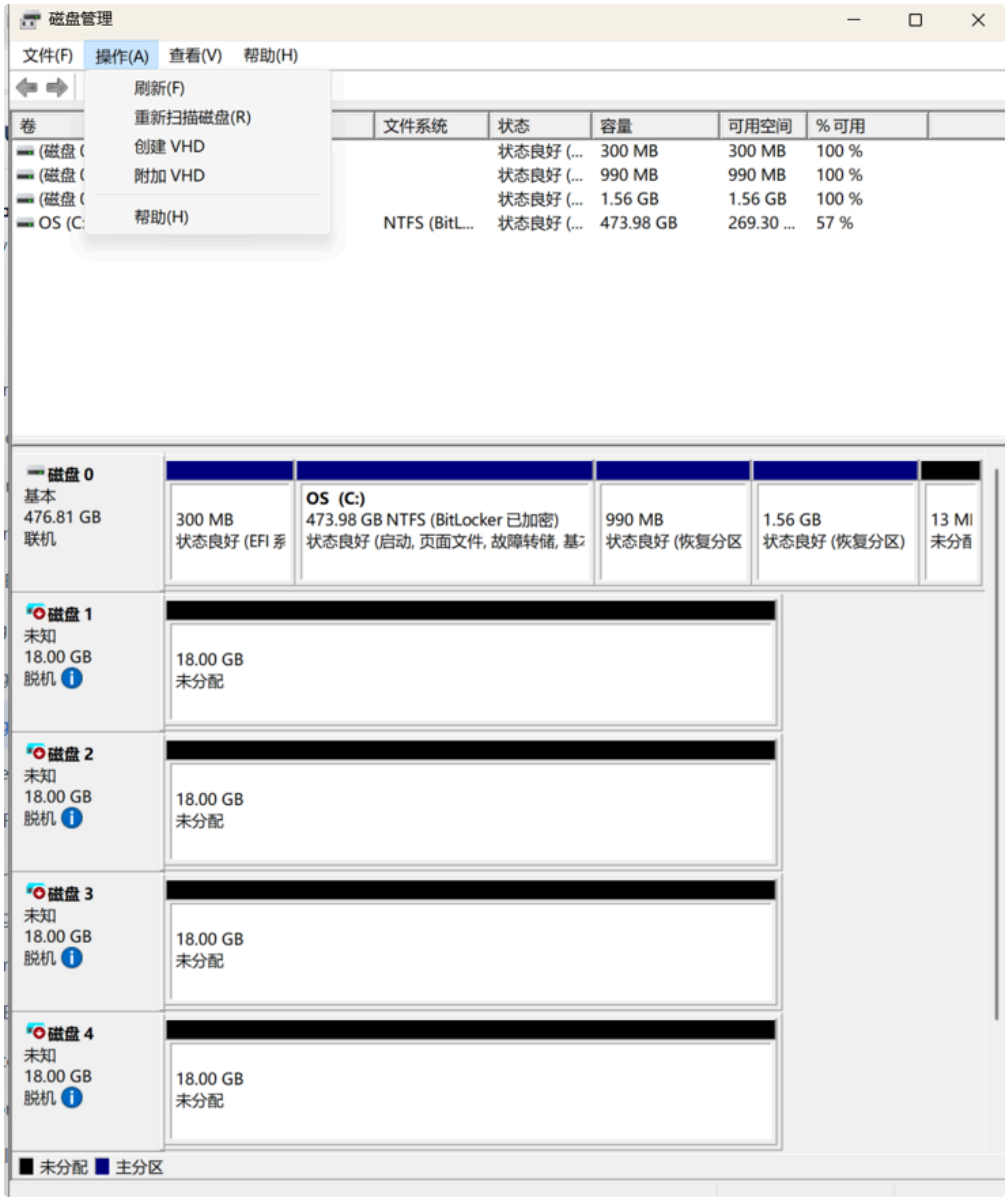
10. Generate at least 4 VHD files as the disks for your VM. [Manage Virtual Hard Disks (VHD) | Microsoft Learn]

a. Make sure all the VHD files are "attached" as WSL/Hyper-V required.

b. Make sure the disks are at least 16GB and use VHDX format if you don't want to allocate all the space at beginning. [How to create virtual drive (VHD, VHDX, Dev Drive) on Windows 11]

[- Pureinfotech]

c. The screenshot of my disk management as below (sorry for the Chinese):



11. Open powershell by administrator and attach the disks to your WSL instance VM. This is not persistent so you need to do this every time your VM restarted: [Get started mounting a Linux disk in WSL 2 | Microsoft Learn]

```
PS C:\Users\rainm> GET-CimInstance -query "SELECT * from Win32_DiskDrive"

DeviceID           Caption                      Partitions Size         Model
--------           -------                      ---------- ----         -----
\\.\PHYSICALDRIVE0 NVMe P0221 NVMe Phison 512GB 4          512105932800 NVMe P0221 NVMe Phison 512GB
\\.\PHYSICALDRIVE1 Microsoft 虚拟磁盘           0          19321182720  Microsoft 虚拟磁盘
\\.\PHYSICALDRIVE2 Microsoft 虚拟磁盘           0          19321182720  Microsoft 虚拟磁盘
\\.\PHYSICALDRIVE3 Microsoft 虚拟磁盘           0          19321182720  Microsoft 虚拟磁盘
\\.\PHYSICALDRIVE4 Microsoft 虚拟磁盘           0          19321182720  Microsoft 虚拟磁盘


PS C:\Users\rainm> wsl --mount \\.\PHYSICALDRIVE1 --bare
PS C:\Users\rainm> wsl --mount \\.\PHYSICALDRIVE2 --bare
PS C:\Users\rainm> wsl --mount \\.\PHYSICALDRIVE3 --bare
```

```
PS C:\Users\rainm> wsl --mount \\.\PHYSICALDRIVE4 --bare
```

12. Now your system is ready for Infinia. You can build and/or install the Infinia. My steps are based on Jason's Running Infinia on Mac with UTM :

    a. Get the codes. I created a branch just in case I need any additional code change.

    ```
    git clone git@github.red.datadirectnet.com:red/red.git
    cd red
    git checkout release/1.3.9 -b sfeng-1.3.9
    ```

    b. Install build environment. I installed docker-compose-v2 to make the "red_build_container.py" work. Then in the following redsetup, it will removed with Infinia's built-in docker-compose

    ```
    sudo apt install docker.io python3-git python3-docker ansible
    ansible-galaxy collection install -r tools/ansible/requirements.yml
    ansible-playbook -c local -i localhost, -b tools/ansible/devenv.yml

    # This will interfere with redsetup later. We can remove it.
    sudo apt remove etcd-server

    # I do this in the first installation to make the build work.
    sudo apt install docker-compose-v2
    # For first installation also
    sudo groupmod docker -a -U $USER
    mkdir /home/$USER/red-pkgs
    ```

    c. Logout and log back in.

    d. Start the build. I choose to upload to a local register.

    ```
    tools/scripts/red_build_container.py -p -rb --registry localhost:5000 -t qrelease,ui --publish-path /home/$USER/red-pkgs --gitrepo .
    ```

    e. Update / install redsetup package, continue following Jason's Running Infinia on Mac with UTM

    ```
    export RED=$HOME/red
    sudo apt purge -y redsetup
    export $(grep ^RED_VERSION $RED/SConstruct | tr -d ' "')
    export BLDTYPE="-engr"
    export ARCH="$(dpkg --print-architecture)"
    export REDSETUP_PKG="redsetup_${RED_VERSION}_${ARCH}${BLDTYPE}.deb"
    sudo apt install -y $RED/tools/packaging/${REDSETUP_PKG} && \
    redsetup --version
    ```

    f. Pull the support containers (we do this since we want to `--skip-image-pull` and use our local image.

    ```
    export REG=harbor.redlab.datadirectnet.com/quay.io && \
    for IMAGE in config events images metrics proxy message healthdb ; do
      docker pull $REG/red/${IMAGE}:1.0.0_${ARCH}
      docker tag $REG/red/${IMAGE}:1.0.0_${ARCH} red/${IMAGE}:1.0.0_${ARCH}
    done

    ```

    g. Start redsetup

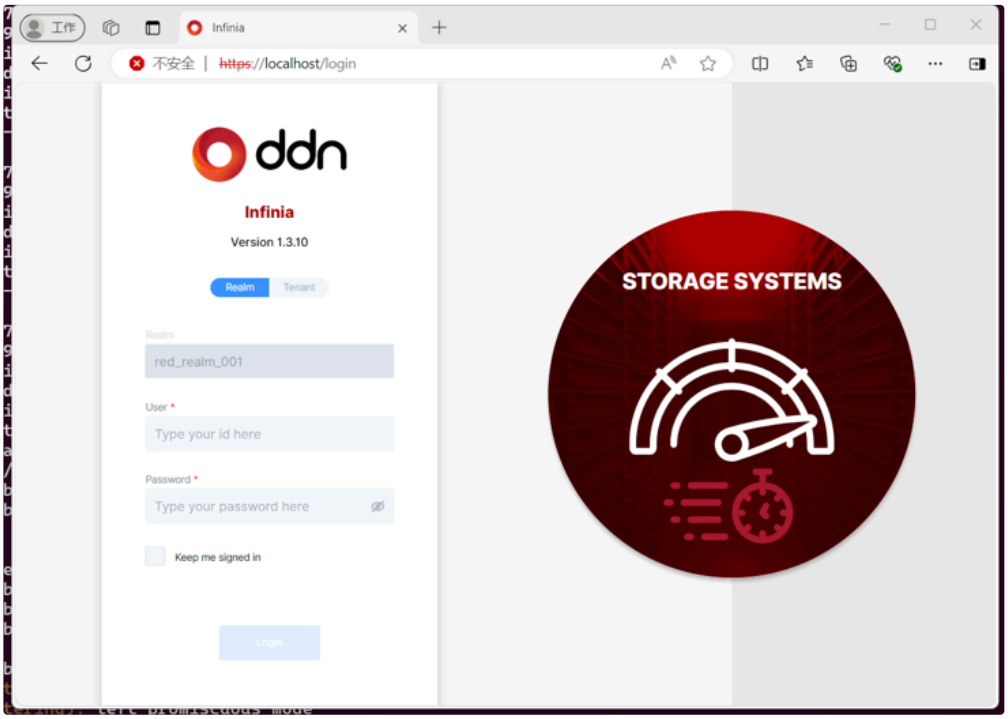
```
export RMD_FILE="rmd_${RED_VERSION}_${ARCH}${BLDTYPE}.json"
echo $RMD_FILE
export MY_IP=$(hostname -I | awk '{print $1}')
echo "My IP address: $MY_IP"

sudo redsetup --realm-entry --realm-entry-secret atleast8 \
  --admin-password atleast8 \
  --skip-image-pull --ctrl-plane-ip $MY_IP \
  --release-metadata-file $RED/tools/packaging/$RMD_FILE \
  --registries 127.0.0.1:5000
```

h. It seems the latest redsetup will not reboot your system after it is done. We still strongly recommend to reboot your node and re-attach all the drives (by above setup #11). This will help SPDK to get enough continuous physical memory.

i. Then you can login to the realm and create your clusters. Make sure your drives are all attached.

```
redcli user login realm_admin -p atleast8
redcli inventory show
redcli realm config update --generate
redcli cluster create cluster1 -z
```

13. You are able to access the WebUI directly from the web browser of your Windows by https://localhost . Refer to [Accessing network applications with WSL | Microsoft Learn]



14. For China developers only, in the step 12.d, the npm build process could be very slow depending on the network connectivity. You can create a proxy for the docker build process (docker build only, NOT docker daemon). Proxy configuration

a. Determine your docker host address. Typically on Linux, it is 172.17.0.1 on the interface “docker0“.

b. Start a proxy listening on the docker host address (not 127.0.0.1, the docker build process is running inside a container, so the 127.0.0.1 will be the containers loopback, not the host's).

c. Create a configure file in your ~/.docker/config.json. My example is:

```
fengshuo@wslserver:~$ cat ~/.docker/config.json
{
 "proxies": {
   "default": {
     "httpProxy": "socks5://172.17.0.1:12345",
     "httpsProxy": "socks5://172.17.0.1:12345",
     "noProxy": "datadirectnet.com,127.0.0.0/8"
   }
 }
}
```

d. Start the build (red_build_container.py) as step 12.d.